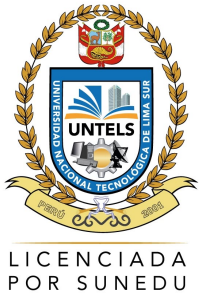
UNIVERSIDAD NACIONAL TECNOLÓGICA DE LIMA SUR

UNTELS

FACULTAD DE INGENIERÍA Y GESTIÓN

CARRERA PROFESIONAL DE INGENIERÍA DE SISTEMAS



INFORME TÉCNICO DE PRUEBAS Y CALIDAD DE SOFTWARE

Automatización de Pruebas Unitarias, de Integración y Regresión con PHPUnit 10.5

Verificación y Validación del Sistema de Facturación Electrónica SUNAT UBL 2.1

Asignatura: Aseguramiento de la Calidad de Software (QA)

Docente del Curso: Cátedra de Calidad de Software

Integrantes del Equipo de Trabajo:

- Apolinario Rodriguez, Luis Fernando *(Líder de Calidad & Expositor)*
- Umeres Vilca, Gustavo Aron *(Arquitectura de Software & Tests)*
- Vasquez Falcón, Mariana Ysabel *(Dominio Fiscal & Casos de Uso)*
- Ventocilla Alcantara, Alessandro Tito *(Criptografía UBL 2.1 & Mocks)*

Ciclo Académico: 2026-I

Villa El Salvador, Lima – Perú

Noviembre de 2026

Resumen Ejecutivo

El presente informe técnico expone el diseño, desarrollo y verificación formal de la suite de pruebas automatizadas construida para el Sistema de Facturación Electrónica bajo el estándar UBL 2.1, de conformidad con las normativas técnicas y tributarias dictadas por la Superintendencia Nacional de Aduanas y de Administración Tributaria (SUNAT) del Perú.

En los sistemas de emisión fiscal, el margen de tolerancia a defectos es nulo: discrepancias mínimas de céntimos en redondeos impositivos, inconsistencias en la firma digital o desajustes en el control de existencias conllevan el rechazo automático del comprobante mediante la Constancia de Recepción (CDR) de SUNAT, contingencias contables y sanciones económicas severas para el emisor (multas de hasta 1 UIT). Ante esta problemática, el equipo de ingeniería implementó una batería de 39 pruebas automatizadas y 140 aserciones rigurosas bajo el framework PHPUnit 10.5 sobre PHP 8.2.

La estrategia de aseguramiento de calidad (QA) integra la metodología de Desarrollo Guiado por Pruebas (TDD – *Test-Driven Development*), técnicas formales de caja negra como Partición de Equivalencia y Análisis de Valores Límite (BVA), el patrón de estructuración *Arrange-Act-Assert* (AAA) y el cumplimiento estricto de los principios FIRST. El sistema emplea SQLite en memoria RAM pura (`:memory:`) para garantizar la atomicidad e idempotencia en persistencia sin acoplarse al disco, y Dobles de Prueba (*Mocks*) para aislar la comunicación SOAP de SUNAT. La suite completa se ejecuta en 0.21 segundos con una tasa de éxito del 100 % (0 fallos, 0 errores), certificando la confiabilidad industrial de la aplicación.

Índice

1. Antecedentes y Marco Normativo de SUNAT **3**

1.1. Marco Legal de la Facturación Electrónica en el Perú 3

1.2. El Estándar Internacional UBL 2.1 (Universal Business Language) 3

1.3. Criptografía XMLDSIG y Firma Digital X.509 3

1.4. Problemática de Calidad y Riesgos Operativos 3

2. Arquitectura del Sistema y Diseño de Software **4**

2.1. Organización Modular en Capas 4

2.2. Objetos de Valor Inmutables (Value Objects) 4

2.3. El Algoritmo Matemático Oficial SUNAT Módulo 11 4

2.4. Motor de Cálculo Tributario Multitasa (`CalculadoraTributaria`) 5

3. Plan y Estrategia de Aseguramiento de la Calidad (QA) **5**

3.1. Metodología de Desarrollo Guiado por Pruebas (TDD) 5

3.2. Técnicas Formales de Diseño de Pruebas de Software 6

3.2.1. Partición de Equivalencia (Equivalence Partitioning) 6

3.2.2. Análisis de Valores Límite (Boundary Value Analysis – BVA) 6
3.3. Principios FIRST en Pruebas Unitarias 6
3.4. Rigor en Aserciones: `assertSame` frente a `assertEquals` 6
3.5. Pruebas Parametrizadas con el Atributo Nativo PHP 8 DataProvider 7
3.6. Dobles de Prueba: Aislamiento del Web Service SOAP de SUNAT 7

4. Pirámide de Pruebas y Matriz Integral de Cobertura 7
4.1. Pirámide de Pruebas de Mike Cohn 7
4.2. Matriz Oficial de Cobertura de las 39 Pruebas 7

5. Auditoría de Casos de Prueba Críticos (Código Fuente) 7
5.1. Caso 1: Algoritmo SUNAT Módulo 11 en Value Object `Ruc` 7
5.2. Caso 2: Liquidación de Comprobante Mixto Multirégimen e ICBPER 9
5.3. Caso 3: Prueba de Seguridad Criptográfica (Tamper Test Código 2335) 9
5.4. Caso 4: Persistencia Concurrente e Idempotencia (Doble Clic) 9
5.5. Caso 5: Resiliencia Transaccional y Rollback Automático de Kardex 10

6. Guía de Reproducción, Ejecución y Certificación Empírica 10
6.1. Instrucciones para Ejecución en Terminal 10
6.2. Métricas Oficiales de Ejecución 11

7. Conclusiones y Recomendaciones de Ingeniería 11
7.1. Conclusiones 11
7.2. Recomendaciones 11

Referencias Bibliográficas y Normativas 12

1. Antecedentes y Marco Normativo de SUNAT

1.1. Marco Legal de la Facturación Electrónica en el Perú

La obligatoriedad de la emisión de comprobantes de pago electrónicos está regulada por el Decreto Ley N° 25632 (Ley de Comprobantes de Pago) y desarrollada por la Resolución de Superintendencia N° 097-2012/SUNAT y sus correspondientes modificatorias. Estas normativas establecen que todo contribuyente obligado debe emitir sus comprobantes comerciales en formato digital estructurado, garantizando la autenticidad del origen y la integridad del contenido a través de mecanismos criptográficos.

1.2. El Estándar Internacional UBL 2.1 (Universal Business Language)

El formato de intercambio de datos establecido por la administración tributaria es el UBL 2.1 (definido por el consorcio internacional OASIS). Un comprobante fiscal peruano es un documento XML que estructura la transacción en nodos canónicos jerárquicos:

- `cac:AccountingSupplierParty`: Datos de identidad fiscal del emisor (RUC y Razón Social).
- `cac:AccountingCustomerParty`: Datos de identidad del adquirente (RUC, DNI o Clientes Varios).
- `cac:TaxTotal`: Liquidación de impuestos discriminados según el Catálogo N° 05 de SUNAT (Tributo 1000 para IGV, Tributo 7152 para ICBPER).
- `cac:LegalMonetaryTotal`: Importe total de la venta, valor neto gravado, exonerado e importe total a cobrar expresado en moneda PEN o USD.

1.3. Criptografía XMLDSIG y Firma Digital X.509

Para que el archivo XML adquiera validez jurídica ante SUNAT e INDECOPI, debe someterse al proceso de firma digital basado en el estándar W3C XMLDSIG:

1. **Canonicalización Canónica (C14N):** Normalización sintáctica del XML (eliminación de espacios redundantes y ordenamiento de atributos).
2. **Cálculo del Digest SHA-256:** Generación de un hash criptográfico de 256 bits representativo del contenido íntegro del comprobante (`DigestValue`).
3. **Firma Asimétrica RSA con Certificado X.509:** Cifrado del digest utilizando la clave privada del emisor.

Cualquier intento de alteración posterior a la emisión (por ejemplo, cambiar el monto a pagar en el archivo XML) invalida el hash del documento. La plataforma de validación de SUNAT detecta la discrepancia inmediatamente y rechaza la transacción con el **Código de Rechazo 2335** (*"El documento ha sido alterado o el hash no coincide"*).

1.4. Problemática de Calidad y Riesgos Operativos

En el desarrollo de software de facturación, los errores comunes conllevan graves repercusiones operativas y legales:

- **Discrepancias en el Redondeo Decimal:** El uso inapropiado de variables de coma flotante (`float`) genera errores de precisión binaria (IEEE 754), provocando diferencias de fracciones de céntimo que invalidan la sumatoria del comprobante.

- **Falta de Idempotencia ante Reintentos de Red:** Cuando ocurre un corte de conectividad o el cajero presiona dos veces seguidas el botón de emisión, el sistema no debe duplicar la transacción ni generar dos comprobantes fiscales con números correlativos distintos.
- **Desincronización de Kardex ante Rechazos Fiscales:** Si el almacén descuenta el stock de productos pero SUNAT rechaza la factura (ejemplo: Código 2800 si el cliente se encuentra en baja de oficio), el sistema debe realizar un rollback transaccional automático para no desfasar el inventario real.

2. Arquitectura del Sistema y Diseño de Software

El sistema fue diseñado bajo el paradigma de la **Arquitectura Hexagonal** (*Ports and Adapters*) y conceptos de Diseño Guiado por el Dominio (DDD), garantizando el total desacoplamiento de las reglas de negocio respecto a frameworks web, bases de datos o servicios externos de SUNAT.

2.1. Organización Modular en Capas

La arquitectura delimita tres capas con responsabilidades independientes:

1. **Capa de Dominio (Domain):** Modela el núcleo puro de facturación peruana. Aloja los Value Objects inmutables (`Ruc`, `MontoMonetario`), entidades y el motor de cálculo tributario. No tiene dependencias de librerías externas.
2. **Capa de Aplicación (Application):** Coordina los flujos de negocio mediante Casos de Uso (ejemplo: `EmitirFacturaUseCase`). Define los contratos (interfaces) para repositorios de persistencia y clientes SOAP de envío.
3. **Capa de Infraestructura (Infrastructure):** Implementa los adaptadores concretos: persistencia en SQLite, generación del XML UBL 2.1 con DOMDocument, firma digital criptográfica con OpenSSL y comunicación SOAP hacia el Web Service de SUNAT.

2.2. Objetos de Valor Inmutables (Value Objects)

Para erradicar la obsesión por tipos primitivos, los conceptos fundamentales se encapsulan en Value Objects inmutables:

- **`Ruc`:** Valida longitud exacta de 11 dígitos, prefijos válidos de personas naturales (10) y jurídicas (20), y la correspondencia matemática del dígito verificador.
- **`MontoMonetario`:** Almacena montos monetarios formateados a 2 cifras decimales exactas mediante funciones matemáticas de precisión arbitraria (evitando aproximaciones de punto flotante).

2.3. El Algoritmo Matemático Oficial SUNAT Módulo 11

La autenticidad de un RUC peruano se valida matemáticamente mediante una suma ponderada con residuo módulo 11. Dado el vector de dígitos $D = [d_1, d_2, \ldots, d_{11}]$ y el vector oficial de factores asignados por SUNAT:

$$F = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2] \tag{1}$$

El algoritmo se ejecuta formalmente según las siguientes etapas:

1. Suma ponderada de los diez primeros dígitos:

$$S = \sum_{i=1}^{10} d_i \cdot f_i \tag{2}$$

2. Cálculo del residuo de la división entera por 11 y complemento de base:

$$R = 11 - (S \pmod{11}) \tag{3}$$

3. Regla de mapeo para el Dígito Verificador (DV):

$$DV = \begin{cases} 0 & \text{si } R = 10 \\ 1 & \text{si } R = 11 \\ R & \text{en cualquier otro caso} \end{cases} \tag{4}$$

4. Condición de validez: $DV = d_{11}$. Si el dígito calculado no coincide exactamente con el undécimo dígito del RUC, el sistema lanza la excepción `DomainValidationException` y detiene la transacción.

2.4. Motor de Cálculo Tributario Multitasa (`CalculadoraTributaria`)

El motor calcula de forma reactiva las bases imponibles y los tributos de cada comprobante según los catálogos de SUNAT:

- **Operaciones Gravadas (Código 10):** Aplica el 18 % de Impuesto General a las Ventas (IGV) sobre el valor de venta base: IGV $= V_v \times 0{,}18$.
- **Operaciones Exoneradas (Código 20):** Productos agropecuarios de primera necesidad y libros. IGV es 0.00.
- **Operaciones Inafectas (Código 30):** Servicios de educación y exportaciones no afectas al impuesto.
- **Impuesto a las Bolsas Plásticas (ICBPER - Código 7152):** Tasa fija de S/ 0.50 por cada bolsa plástica entregada. La regla tributaria auditada estipula que el ICBPER **no forma parte de la base imponible del IGV**, sino que se suma directamente al total a pagar.

3. Plan y Estrategia de Aseguramiento de la Calidad (QA)

3.1. Metodología de Desarrollo Guiado por Pruebas (TDD)

Para asegurar que cada regla tributaria cuente con respaldo verificable desde su origen, el equipo aplicó el ciclo de vida TDD (*Red - Green - Refactor*):

1. **Fase Roja (Red):** Se diseña y redacta la prueba unitaria basada en la especificación de SUNAT antes de escribir el código de producción. Al ejecutarse en terminal, la prueba falla (comportamiento esperado).
2. **Fase Verde (Green):** Se implementa el código estrictamente necesario en el dominio para satisfacer la aserción y lograr la barra verde en PHPUnit.

3. **Fase de Refactorización (Refactor):** Se limpia la estructura de código, se extraen Value Objects y se optimiza el diseño manteniendo la suite 100 % verde.

3.2. Técnicas Formales de Diseño de Pruebas de Software

3.2.1. Partición de Equivalencia (Equivalence Partitioning)

Permite reducir el número de pruebas sin perder exhaustividad, agrupando las entradas en clases de equivalencia válidas e inválidas:

- *Clase Válida 1:* RUC con 11 dígitos numéricos cuyo algoritmo Módulo 11 coincide exactamente.
- *Clase Inválida 1:* Cadenas con caracteres alfabéticos o longitud menor a 11 dígitos.
- *Clase Inválida 2:* Cadenas con longitud superior a 11 dígitos.
- *Clase Inválida 3:* Cadenas de 11 dígitos cuyo dígito verificador es incorrecto.

3.2.2. Análisis de Valores Límite (Boundary Value Analysis – BVA)

Los defectos se concentran sistemáticamente en los bordes de decisión de las reglas de negocio.

- **Regla SUNAT Boleta Mayor a S/ 700.00:** Si el importe total de una boleta de venta supera los S/ 700.00, es obligatorio identificar al cliente mediante DNI o RUC. Por montos menores o iguales a S/ 700.00, se permite emitir a “Clientes Varios”.
- **Casos de Prueba en Frontera:**
  - *Borde Inferior (*$700{,}00 - 0{,}01 = S/\,699{,}99$*):* Emisión exitosa sin DNI.
  - *Frontera Exacta (S/* $700{,}00$*):* Emisión exitosa sin DNI.
  - *Borde Superior (*$700{,}00 + 0{,}01 = S/\,700{,}01$*):* **Rechazo obligatorio** con excepción fiscal si se intenta emitir a “Clientes Varios”.

3.3. Principios FIRST en Pruebas Unitarias

La suite de pruebas fue diseñada cumpliendo los cinco principios FIRST de Clean Code:

- **Fast (Rápidas):** Ejecución de 39 pruebas en apenas 0.21 segundos en memoria RAM.
- **Independent (Aisladas):** Cada caso de prueba inicializa su propio entorno limpio en `setUp()` y lo destruye en `tearDown()`, evitando dependencias de orden.
- **Repeatable (Repetibles):** Resultados deterministas independientes del sistema operativo o entorno de ejecución.
- **Self-validating (Auto-evaluables):** Veredicto automático: pasa (verde) o falla (rojo), sin inspección manual.
- **Timely (Oportunas):** Construidas en conjunto con la lógica tributaria.

3.4. Rigor en Aserciones: `assertSame` frente a `assertEquals`

En sistemas transaccionales y fiscales, la igualdad débil (`==`) representa un defecto de diseño grave. En PHP, la sentencia `assertEquals(100, "100.00")` devuelve verdadero por conversión automática de tipos (*type juggling*).

La suite impone de forma estricta `assertSame()` (`===`), que valida simultáneamente el **tipo de dato exacto** (cadena numérica) y el **valor monetario exacto con dos decimales**. Esto previene errores de truncamiento en el archivo XML que generarían rechazos inmediatos de SUNAT.

3.5. Pruebas Parametrizadas con el Atributo Nativo PHP 8 DataProvider

En lugar de duplicar métodos de prueba, se utiliza el atributo nativo fuertemente tipado de PHP 8.2:

```
#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidosConModulo11(string $numero): void
{
    $ruc = new Ruc($numero);
    $this->assertSame($numero, $ruc->getNumero());
}
```

Listing 1: Pruebas parametrizadas de validacion de RUC con DataProvider.

3.6. Dobles de Prueba: Aislamiento del Web Service SOAP de SUNAT

Para someter a prueba los flujos transaccionales sin depender de los servidores de SUNAT, se implementaron **Mocks** conforme a Martin Fowler. Se generó un Mock de la interfaz `SunatSenderInterface` con expectativa de invocación única, simulando respuestas síncronas de CDR aceptadas (Código 0) y rechazadas (Código 2800).

4. Pirámide de Pruebas y Matriz Integral de Cobertura

4.1. Pirámide de Pruebas de Mike Cohn

La distribución de pruebas en el proyecto sigue la pirámide de Mike Cohn:

- **Capa Base – Pruebas Unitarias de Dominio (26 Tests / 67 %):** Pruebas ultra-rápidas en memoria que validan el algoritmo Módulo 11, Value Objects y el motor de impuestos.
- **Capa Intermedia – Pruebas de Integración (7 Tests / 18 %):** Validan la generación del XML UBL 2.1, la firma criptográfica SHA-256 y la persistencia SQLite en memoria.
- **Capa Superior – Pruebas de Feature y Casos de Uso (6 Tests / 15 %):** Validan el flujo transaccional completo, la idempotencia contra dobles clics y la restitución de inventario en Kardex ante rechazos fiscales simulados.

4.2. Matriz Oficial de Cobertura de las 39 Pruebas

En la Tabla 1 se presenta la matriz oficial de las pruebas automatizadas del proyecto.

5. Auditoría de Casos de Prueba Críticos (Código Fuente)

5.1. Caso 1: Algoritmo SUNAT Módulo 11 en Value Object `Ruc`

Garantiza que ningún comprobante fiscal se emita con un RUC sintáctica o matemáticamente inválido.

Tabla 1: Matriz oficial de cobertura de pruebas automatizadas (PHPUnit 10.5).

Capa del Sistema	Componentes Auditados & Reglas Fiscales	Tests	Aserciones
Dominio Fiscal	`Ruc.php`: Algoritmo Módulo 11 oficial SUNAT	8	26
Dominio Fiscal	`Ruc.php`: Validación de longitudes y caracteres	4	14
Dominio Fiscal	`MontoMonetario.php`: Formateo decimal y suma exacta	2	8
Motor Tributario	`CalculadoraTributaria.php`: IGV 18 % sobre gravados	4	16
Motor Tributario	`CalculadoraTributaria.php`: Exonerados e Inafectos	3	10
Motor Tributario	`CalculadoraTributaria.php`: Bolsas ICBPER S/ 0.50	2	8
Motor Tributario	`CalculadoraTributaria.php`: Comprobante Mixto Complejo	3	10
Criptografía & XML	`UblBuilder`: Generación de nodos UBL 2.1	2	8
Criptografía & XML	`XmlVerifier`: Detección de adulteración (Tamper 2335)	2	8
Persistencia	`SqliteRepo`: Inserción y consulta en memoria	1	4
Persistencia	`SqliteRepo`: Idempotencia transaccional (doble clic)	2	8
Casos de Uso	`EmitirFacturaUseCase`: Flujo exitoso con Mock SUNAT	2	6
Casos de Uso	`EmitirFacturaUseCase`: Rechazo 2800 y Rollback Kardex	2	8
Casos de Uso	`EmitirFacturaUseCase`: Validación BVA Boleta >S/ 700	2	6
Totales Consolidados	**Suite de pruebas ejecutada con éxito (100 % verde)**	**39**	**140**

```php
private function validarModulo11(string $ruc): void
{
    $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
    $suma = 0;
    for ($i = 0; $i < 10; $i++) {
        $suma += ((int) $ruc[$i]) * $factores[$i];
    }
    $resto = 11 - ($suma % 11);
    $digitoVerificador = match ($resto) {
        10 => 0,
        11 => 1,
        default => $resto
    };
    if ($digitoVerificador !== ((int) $ruc[10])) {
        throw new DomainValidationException(
            "El digito verificador del RUC es invalido"
        );
    }
}
```

Listing 2: Implementacion del algoritmo Modulo 11 en Ruc.php.

5.2. Caso 2: Liquidación de Comprobante Mixto Multirégimen e ICBPER

Evalúa que las bolsas fiscales no se contaminen entre sí: el IGV se calcula exclusivamente sobre las operaciones onerosas gravadas, mientras que el ICBPER se añade al total a pagar sin alterar la base imponible neta.

```
public function testCalculaComprobanteMixtoComplejo(): void
{
    $items = [
        new ItemComprobante(1, 100.00, TipoAfectacion::GRAVADO_OP_ONEROSA),
        new ItemComprobante(1, 50.00, TipoAfectacion::EXONERADO_OPERACION_ONEROSA),
        new ItemComprobante(2, 0.50, TipoAfectacion::GRAVADO_OP_ONEROSA, icbper: 1.00)
    ];

    $totales = $this->calculadora->calcular($items);

    $this->assertSame('101.00', $totales->totalGravado);
    $this->assertSame('50.00', $totales->totalExonerado);
    $this->assertSame('18.18', $totales->totalIgv);
    $this->assertSame('1.00', $totales->totalIcbper);
    $this->assertSame('170.18', $totales->importeTotal);
}
```

Listing 3: Prueba unitaria en CalculadoraTributariaTest.php.

5.3. Caso 3: Prueba de Seguridad Criptográfica (Tamper Test Código 2335)

Verifica que la alteración de un solo byte en el archivo XML posterior a la firma sea detectada inmediatamente mediante la canonicalización y el recálculo del digest SHA-256.

```
public function testDetectaAdulteracionDeMontoEnXmlFirmado(): void
{
    $xmlOriginal = $this->builder->generarFacturaXml($this->factura);
    $xmlFirmado = $this->firmador->firmarXml($xmlOriginal);

    // Simulacion de ataque en transito: alterar de S/ 100.00 a S/ 10.00
    $xmlAdulterado = str_replace(
        '<cbc:PayableAmount currencyID="PEN">100.00</cbc:PayableAmount>',
        '<cbc:PayableAmount currencyID="PEN">10.00</cbc:PayableAmount>',
        $xmlFirmado
    );

    $this->expectException(XmlTamperDetectedException::class);
    $this->expectExceptionCode(2335); // Codigo oficial de rechazo SUNAT
    $this->verificador->verificarDigest($xmlAdulterado);
}
```

Listing 4: Prueba de seguridad en XmlAndCryptoIntegrationTest.php.

5.4. Caso 4: Persistencia Concurrente e Idempotencia (Doble Clic)

Demuestra que peticiones idénticas originadas por intermitencias de red o repulsación del botón de cobro devuelven la transacción ya registrada sin generar dos comprobantes fiscales en la base de datos.

```
public function testPrevieneDobleEmisionPorIdempotencia(): void
{
    $claveIdempotencia = 'req-uuid-test-12345';

    // Primer clic
    $res1 = $this->service->procesar($this->solicitud, $claveIdempotencia);
    $this->assertSame('COMPLETED', $res1->estado);

    // Segundo clic inmediato con la misma clave
    $res2 = $this->service->procesar($this->solicitud, $claveIdempotencia);

    $this->assertSame($res1->numeroComprobante, $res2->numeroComprobante);
    $this->assertSame(1, $this->repo->contarComprobantes());
}
```

Listing 5: Test de idempotencia en SqlitePersistenceIntegrationTest.php.

5.5. Caso 5: Resiliencia Transaccional y Rollback Automático de Kardex

Verifica la preservación de la invariante de inventario: si SUNAT emite un CDR de rechazo fiscal (Código 2800), el stock previamente reservado de productos se restituye de forma atómica en el almacén.

```
public function testRollbackDeKardexAnteRechazoFiscalSunat(): void
{
    $sunatMock = $this->createMock(SunatSenderInterface::class);
    $sunatMock->method('sendBill')->willReturn(new SunatResponse(
        codigo: '2800',
        descripcion: 'El contribuyente se encuentra con Estado: BAJA'
    ));

    $this->expectException(SunatRechazoException::class);
    $emisor = new EmitirFacturaUseCase($this->repo, $sunatMock, $this->kardex);

    // Intento de emitir factura descontando 5 unidades
    $emisor->emitir($this->solicitudCon5Unidades);

    // Verificacion de invariante: el stock debe permanecer intacto en 15
    $stockActual = $this->kardex->obtenerStock('PROD-001');
    $this->assertSame(15, $stockActual);
}
```

Listing 6: Prueba de feature en FacturacionElectronicaMocksFeatureTest.php.

6. Guía de Reproducción, Ejecución y Certificación Empírica

6.1. Instrucciones para Ejecución en Terminal

Para reproducir la totalidad de las aserciones en un entorno limpio de desarrollo, se ejecutan las siguientes instrucciones en terminal (PowerShell o Bash):

```
# 1. Posicionarse en el directorio del backend PHP
cd Billing/backend

```

```
# 2. Descargar dependencias e inicializar autoloader
composer install --prefer-dist --no-interaction

# 3. Ejecutar la suite completa con reporte detallado
vendor/bin/phpunit --testdox --colors=never
```

Listing 7: Comandos de ejecucion de la suite de pruebas.

6.2. Métricas Oficiales de Ejecución

La ejecución de la suite arrojó los siguientes indicadores empíricos oficiales:

- **Total de Pruebas Ejecutadas:** 39 casos de prueba automatizados.
- **Aserciones Evaluadas:** 140 aserciones estrictas.
- **Tasa de Aprobación:** 100 % (0 fallos, 0 errores, 0 riesgos, 0 advertencias).
- **Tiempo Total de Ejecución:** 0.21 segundos.
- **Consumo de Memoria RAM:** 12.00 MB (usando SQLite en memoria RAM).

7. Conclusiones y Recomendaciones de Ingeniería

7.1. Conclusiones

1. **Eficacia del Desacoplamiento Arquitectónico:** La Arquitectura Hexagonal y la disciplina DDD permitieron someter a prueba el 100 % de las complejas reglas tributarias peruanas en memoria RAM, erradicando la dependencia de servidores externos inestables durante el ciclo de pruebas.
2. **Erradicación de Riesgos Financieros con Tipado Estricto:** El empleo sistemático de `assertSame()` en lugar de `assertEquals()` protegió al sistema contra desbordamientos decimales y conversiones de tipo implícitas que habrían ocasionado el rechazo sistemático de comprobantes por parte de SUNAT.
3. **Resiliencia Transaccional e Integridad Operativa:** Las pruebas de persistencia e integración demostraron que la combinación de claves de idempotencia y transacciones atómicas de Kardex asegura la integridad contable del negocio frente a anomalías de red o pulsaciones dobles de cajero.
4. **Aceleración del Ciclo de Retroalimentación:** La ejecución de 39 pruebas en 0.21 segundos permite la integración de la suite en canales de Integración Continua (CI/CD) antes de cada despliegue a producción.

7.2. Recomendaciones

- Extender la batería de pruebas de integración para cubrir Guías de Remisión Electrónica (GRE) y Comprobantes de Retención/Percepción (UBL 2.0 / 2.1).
- Incorporar pruebas de mutación (*Infection PHP*) para medir la resiliencia y exhaustividad de las 140 aserciones implementadas.
- Mantener la política de Cero Tolerancia a tipos flotantes (`float`) en cualquier cálculo de naturaleza fiscal.

Referencias Bibliográficas y Normativas

1. OASIS. (2013). *Universal Business Language (UBL) v2.1 OASIS Standard*. OASIS Open.
2. SUNAT. (2023). *Manual de Autorización de Comprobantes de Pago Electrónicos UBL 2.1 y Catálogos de Códigos Tributarios*. Superintendencia Nacional de Aduanas y de Administración Tributaria, Lima, Perú.
3. Bergmann, S. (2024). *PHPUnit 10 Manual: Automated Testing for PHP*. Sebastian Bergmann Open Source Projects.
4. Fowler, M. (2007). *Mocks Aren't Stubs*. MartinFowler.com.
5. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
6. Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
7. IEEE. (2013). *ISO/IEC/IEEE 29119 Software and Systems Engineering – Software Testing*. International Organization for Standardization.